The Advance Directive Ontology (ADO): A Computable Representation of End-of-Life Care Preferences in Advanced Heart Failure

Patrick Walsh, B.S.[1] **Darren Chan, B.S.**[1]

[1]Stanford University, Stanford, CA, USA

Abstract

End-of-life care preferences are recorded in instruments—advance health care directives (AHCDs), POLST forms, and code-status orders—that are low-resolution or disease-blind and support no automated reasoning over conditional, graded, negatable preferences. We present the Advance Directive Ontology (ADO), an OWL 2 ontology (67 classes, 22 properties) scoped to five advanced heart failure decision points (CPR, ventilation, ICD/device management, vasopressor/inotrope escalation, dialysis) with 21 intervention subclasses. A reasoner matches a patient's encoded preferences against a clinical scenario and returns one of four outputs (clear, partial, no coverage, vague), preserving original language. The reasoner scored 16/16 on clinical vignettes, and on 12 directive profiles reproduced clinician-standard hospital code status and POLST orders with 97% field agreement. A closed-world large-language-model pipeline extracts preferences from real directive text at F1 0.97, inventing nothing for out-of-scope statements. Expert review reframed ADO as a decision-support aid for advance-care-planning documentation—not a legal directive.

Introduction

Patients record end-of-life (EOL) care preferences through several distinct instruments, each occupying a different layer of the care pathway: advance health care directives (AHCDs) capture durable values and wishes; Physician Orders for Life-Sustaining Treatment (POLST) translate them into portable medical orders; and clinician-entered code-status orders (full-code, do-not-resuscitate, do-not-intubate, do-not-escalate) are what is actually acted on at the bedside during an acute crisis. An estimated 67% of intensive-care patients lack decision-making capacity at EOL,[1] yet fewer than 37% of US adults have any directive on file,[2] and the documents that do exist are rarely interoperable with electronic health record (EHR) systems.[3]

A common framing holds that the core problem is that directives are unstructured free text. This is only partly true: the better AHCDs—such as the California statutory form and the University of California directive—are largely structured checkbox instruments. The more fundamental problem is that these instruments are **low-resolution and disease-blind**. Checkbox forms collapse conditional, graded preferences ("I would accept a ventilator temporarily if there is a reversible cause, but never indefinitely") into a few coarse options, and they omit almost entirely the device- and disease-specific decisions that dominate advanced heart failure (HF): implantable cardioverter-defibrillator (ICD) deactivation, left-ventricular-assist-device (LVAD) management, and inotrope escalation. Approximately 200,000 HF patients in the US have ICDs,[4] yet most generic AD templates do not mention them.

Heart failure is a revealing scope for this problem. Its trajectory is unpredictable, oscillating between stable and acute states so that "terminal" is hard to define; it is device-dependent in ways generic templates never anticipated; and patients cycle repeatedly through the emergency department and ICU, generating frequent, time-pressured preference-interpretation events.[5] Crucially, no published OWL ontology, to our knowledge, supports description-logic reasoning over conditional, negatable, graded EOL preferences. Adjacent work occupies neighboring layers: the HL7 PACIO Advance Directive Interoperability guide[3] transports directive *documents* as FHIR resources but does not reason over their content; POLST encodes *orders* as binary checkboxes that lose conditionality; and the Stanford Letter Project[6] preserves narrative voice but is not computable. ADO targets the missing *inference* layer above these.

Objectives

We set out to (1) build a formal, computable ontology of EOL care preferences for advanced HF; (2) provide a pipeline that encodes patient preferences as ontology individuals; (3) implement a reasoner that matches encoded preferences to a clinical scenario and returns a clinically honest output that flags uncertainty rather than forcing an answer; (4) demonstrate that the same pipeline can ingest free-text directives via a large language model (LLM) constrained to closed-world extraction; and (5) subject the design to expert clinical-ethics review. We emphasize at the outset—a point reinforced by that review (below)—that ADO is a **decision-support and documentation aid**, intended to help pre-populate an advance-care-planning (ACP) progress note. It is explicitly *not* a legal advance directive, and its output carries no statutory force and does not replace an AHCD, POLST, or code-status order.

Methods

Advance directive concept inventory

To ground the ontology in real-world document diversity, we compiled and cataloged 50 advance directive templates across 8 categories (Table 1), drawing on state statutory forms, national repositories (CaringInfo,[12] National POLST[13]), and specialized populations. Preliminary analysis confirmed our central hypothesis: the HF-specific decisions we target (ICD deactivation, inotrope/vasopressor escalation) are almost entirely absent from standard AD templates.

Table 1. Advance directive concept inventory by category.

Category	Count	Examples
State-specific ADs (US)	10	California AHCD, Texas Directive, NY Health Care Proxy
POLST/MOLST forms	6	National POLST, CA POLST, NY MOLST
Specialized populations	4	Five Wishes, Voicing My Choices, Dementia-specific
Psychiatric/mental health	7	Bazelon Center PAD, SAMHSA guide
Religious/faith-based	9	Catholic (NCBC), Halachic, Islamic (IMANA)
Organizational	4	VA Form 10-0137, AAFP, Mayo Clinic
DNR/DNAR forms	5	Texas OOH-DNR, CA Prehospital DNR
International	5	UK ADRT (NHS), BC "My Voice"
Total	**50**	

Ontology design

The ADO contains 67 classes, 13 object properties, and 9 data properties, authored in Protégé[8] using OWL 2[7] for its formal description-logic semantics and reasoner compatibility. It is scoped to the five HF decision points where directives are most often consulted—CPR/resuscitation, mechanical ventilation, ICD/device management, vasopressor/inotrope escalation, and dialysis—with 21 specific intervention subclasses. Each preference is a `PreferenceStatement` linking an `Intervention` to an `ActivationCondition` (NYHA functional class,[10] clinical condition, reversible-cause flag, care context, and time bound), a `PreferenceStrength` (Absolute / Strong / Conditional / Weak), an `isNegated` boolean, and the patient's verbatim language via `originalText`. Clinical concepts are grounded in SNOMED CT[11] for interoperability. Two design choices are central:

- **Vagueness as a first-class category.** A `VaguePreference` subclass preserves irreducibly imprecise patient language ("no heroic measures") via `originalText` while assigning it to the nearest formal class, so the system acknowledges what it cannot fully represent rather than forcing false precision.
- **Negation under the open-world assumption.** Exclusionary preferences are encoded with the `isNegated` flag and, where needed, `complementOf` class expressions—a deliberate accommodation of OWL's open-world semantics.[7]

Preference-input and reasoning pipeline

A Python pipeline (`preference_input.py`, built on owlready2[9]) instantiates OWL individuals from structured JSON, validating inputs against controlled vocabularies for all 21 interventions, clinical conditions, NYHA classes, care contexts, and document types. It supports interactive CLI entry, JSON-file batch input, and example generation. A scenario-based query engine (`query_evaluation.py`) then takes a populated patient ontology plus a clinical scenario, matches each preference's activation conditions against the scenario state, and classifies the result into one of four outputs (Table 2). The 4-category output is the central design choice: it is meant to be clinically honest, flagging uncertainty rather than forcing an answer. Temporal conditions are handled by storing `hasTimeBoundHours` as a numeric property so that, e.g., "withdraw vasopressors if no improvement after 72 h" resolves by direct comparison once the elapsed time is known; care-context conditions are handled by a `requiresCareContext` property so that a hospice-conditional preference does not fire in the ICU.

Closed-world LLM extraction

To show that ADO can ingest real free-text directives, a pipeline (`llm_extraction.py`) uses a large language model (Claude, `claude-opus-4-7`) with a Pydantic-typed schema to convert a verbatim directive paragraph into the same JSON the structured pipeline already accepts. The system prompt enforces **closed-world extraction**: the model encodes *only* what the patient explicitly stated and never extrapolates, because the reasoner's "no coverage" behavior depends

Table 2. The reasoner's four-category output.

Output	Meaning
Clear match	Activation conditions met; preference unambiguous
Partial match	Some conditions met, others unspecified or explicitly unmet—defer to surrogate
No coverage	The directive does not address this scenario
Vague match	Preference uses irreducibly imprecise terms; original language surfaced with nearest formal class

on that discipline. The ontology, not the LLM, remains the auditable source of truth; the LLM is glue at the boundary, not the reasoning core.

Code-status and POLST derivation

Motivated by expert review (below)—that the bedside is governed by code-status orders and POLST, not the directive—we built a layer (`code_status.py`) that aggregates intervention-level preferences into a **hospital code status** (Full code / DNR / DNI / DNR+DNI, with an optional "Do Not Escalate" qualifier), a **POLST Section A** order (Attempt CPR / DNAR), and a **POLST Section B** treatment level (Full / Selective / Comfort-Focused). Code status and Section A are scenario-grounded, so a conditional refusal yields DNR only when its conditions are met—and the system flags that conditionality, which a flat checkbox cannot express; Section B reflects standing willingness read from the preference set.

Evaluation design

We evaluated the reasoner on (1) 16 clinical vignettes spanning all five decision points and all four preference types (clear, conditional, negated, vague), each comparing the system's decision and match type against a human-defined expected answer; and (2) a *code-status mapping* task over 12 preference profiles grounded in real inventory templates (e.g., California AHCD, Texas out-of-hospital DNR, NHS Wales ADRT, Five Wishes, VA form, dementia directive), scoring ADO's derived code status and POLST orders against a gold standard adjudicated from POLST's published definitional semantics—deliberately independent of ADO's own logic; and (3) an *LLM extraction* task over 12 statements drawn from real directive template families (CA statutory AHCD, Texas out-of-hospital DNR, common living wills, Five Wishes-style comfort language, VA full-treatment language, an ICD/hospice clause, HF-specific escalation), with two deliberately out-of-scope statements (artificial nutrition/hydration; antibiotics) to test the closed-world discipline. We additionally subjected the design to expert review by Dr. David Magnus, Director of the Stanford Center for Biomedical Ethics, via a written packet and a structured meeting.

Results

Pipeline validation

An example patient with 9 preference statements—spanning all five decision points, clear and conditional, negated and affirming, across strength levels—was instantiated end-to-end without warnings or class-not-found errors, producing a populated ontology of 50 individuals.

Scenario-based reasoning

The reasoner achieved **16/16 (100%) decision accuracy and 16/16 (100%) match-type accuracy** across the vignette suite (Table 3), correctly handling all four output categories, temporal thresholds in both the "fires" and "does-not-fire-yet" directions (V10/V13), care-context gating (V5/V6), etiology-specific conditions (V14), and intervention granularity that prevents over-inference (V16). Two earlier failure modes documented in our progress report—a missing care-context property and a string-typed time bound—were resolved by the `requiresCareContext` and numeric `hasTimeBoundHours` additions.

Free-text to decision via the LLM pipeline

We gave the LLM a verbatim directive paragraph; it extracted 8 structured preferences with original language preserved, which we piped into a fresh ontology and queried for CPR in a NYHA-IV cardiac arrest. The reasoner returned *no / clear* with all four activation conditions matching. The first run instead returned *vague*, because the model had faithfully captured a second, vague CPR-relevant preference ("no heroic measures if I am dying") with its "if I am dying" qualifier as an explicit, unmet `TerminalCondition`—and we had not yet written a precedence rule for a clear specific preference

Table 3. Clinical vignette evaluation. Match types: clear (9), partial (4), no coverage (2), vague (1).

ID	Clinical scenario	Intervention	Expected	System	Result
V1	Cardiac arrest, NYHA IV, no reversible cause	CPR	No (clear)	No (clear)	PASS
V2	Cardiac arrest, NYHA III, reversible cause	CPR	Partial	Partial	PASS
V3	Resp. failure, reversible cause	Temp. ventilation	Yes (clear)	Yes (clear)	PASS
V4	Ventilator 10d, no improvement	Indef. ventilation	No (clear)	No (clear)	PASS
V5	Enrolled in hospice	ICD deactivation	Yes (clear)	Yes (clear)	PASS
V6	In ICU, not hospice	ICD deactivation	Partial	Partial	PASS
V7	Cardiorenal syndrome, recovery expected	Acute dialysis	Yes (clear)	Yes (clear)	PASS
V8	Kidney failure, no recovery	Chronic dialysis	No (clear)	No (clear)	PASS
V9	Cardiogenic shock	Inotrope escalation	Yes (clear)	Yes (clear)	PASS
V10	Cardiogenic shock, vasopressors 96 h	Vasopressor w/d	Yes (clear)	Yes (clear)	PASS
V11	Respiratory failure	NIV (BiPAP)	Vague	Vague	PASS
V12	Stable, NYHA II	Pacemaker deactivation	No coverage	No coverage	PASS
V13	Cardiogenic shock, vasopressors 48 h	Vasopressor w/d	Partial	Partial	PASS
V14	Sepsis-AKI (not cardiorenal), recovery expected	Acute dialysis	Partial	Partial	PASS
V15	Stable, outpatient dialysis consult	Chronic dialysis	No (clear)	No (clear)	PASS
V16	VT, ICD about to shock	ICD shock therapy	No coverage	No coverage	PASS

versus a vague preference with explicit but unmet conditions. We named and codified that rule (a vague preference whose own activation conditions are explicitly unmet does not fire) and re-ran. This is the auditability argument made concrete: the structured layer forced an implicit assumption to become an explicit, documented rule—something a pure-LLM system would have hidden behind confident prose.

Extraction accuracy and closed-world discipline

Over the 12 real-template statements (15 in-scope gold preferences), the extractor achieved **precision 0.94, recall 1.00, F1 0.97**, with negation, preference type (clear/conditional/vague), and conditionality each correct on **15/15** matched preferences. Most important for the architecture's premise, it committed **zero over-extractions on the two out-of-scope statements**: given text about artificial nutrition/hydration and antibiotics—interventions outside our vocabulary—it extracted nothing rather than inventing a mapping into the nearest covered class. This is precisely the discipline the reasoner's "no coverage" output depends on. The single false positive (the only departure from perfect precision) was a defensible over-split: the vague "no heroic measures or to be kept alive by machines if I am dying" was mapped to two vague interventions (CPR and mechanical ventilation) where the gold expected one.

Code-status / POLST mapping (the bedside test)

Across the 12 directive profiles, ADO's derived code status and POLST orders agreed with the POLST-semantics gold standard on **35/36 fields (97%)**, with **11/12** profiles matching on all three fields (Table 4). The hospital code-status confusion matrix was fully diagonal across all six observed categories (Full code, DNR, DNI, DNR/DNI, Do Not Escalate, DNR/DNI+Do Not Escalate); chance-corrected agreement with the gold standard was Cohen's $\kappa = 1.00$ for code status and POLST Section A, and $\kappa = 0.88$ for Section B. Two results matter beyond the pass rate. First, four profiles produced a code status ADO marks as *conditional*: profiles P9 and P10 encode the same directive ("no CPR if NYHA IV with no reversible cause") and correctly flip between DNR and Full code as the scenario meets or fails that condition—precisely the conditionality a flat POLST checkbox discards. Second, the single divergence (P5) is principled rather than erroneous: when a directive's activation condition (permanent unconsciousness) is not met, ADO returns POLST Section B = *Not specified* instead of presuming *Full Treatment* as the POLST default does—an honest refusal to presume a treatment level the directive never addressed.

Table 4. Code-status / POLST mapping fidelity across 12 directive profiles (gold standard from POLST semantics).

Metric	Agreement
Hospital code status	12/12 (100%)
POLST Section A (resuscitation)	12/12 (100%)
POLST Section B (treatment level)	11/12 (92%)
Exact-profile match (all three fields)	11/12 (92%)
Overall field agreement	**35/36 (97%)**

Decision-point coverage

For the five in-scope decision points the ontology provides 21 intervention classes—more granular than any single template in the inventory (standard ADs offer a binary on "mechanical ventilation"; ADO distinguishes six ventilation-related interventions). Table 5 summarizes coverage and the gaps surfaced by expert review.

Table 5. Ontology coverage of common decision points.

Decision point	Coverage	Notes
CPR / resuscitation	Full (3 subclasses)	CPR, defibrillation, transcutaneous pacing
Mechanical ventilation	Full (6 subclasses)	Temporary vs. indefinite, invasive vs. non-invasive, withdrawal
ICD / device mgmt	Full (5 subclasses)	ICD deactivation, shock therapy, LVAD, pacemaker
Vasopressor / inotrope	Full (4 subclasses)	Escalation and withdrawal for both
Dialysis	Full (3 subclasses)	Acute, chronic, withdrawal
Artificial hydration/nutrition	Not yet covered	Flagged by expert review as a priority gap; addressed by POLST
Antibiotics, comfort care	Not covered	Future work

Expert review

Dr. Magnus materially reframed the project. His central point was that the system's output "can't [be thought of] as AHCDs from a legal perspective. The patient has not signed anything and it likely does not meet the regulatory requirements. This is more like a progress note." He challenged the free-text framing directly—"many (most?) AHCD directives … have more or less designed them as check boxes … it isn't largely a problem of free text"—which sharpened our motivation to the resolution-and-coverage argument above. His deepest critique was dimensional: "one problem with this system is that it is one dimensional. But patient preferences are often two or more dimensional. E.g., I don't want to be intubated, but I want my family to be comfortable … so do what they say." Under the law a capacitous patient's preferences govern, but he noted that studies show most patients prefer that families be allowed to override. He flagged two concrete omissions—artificial hydration/nutrition and the POLST concept of a *time-limited trial*—and was reassuring on liability ("liability exists no matter what you do … suits are rare anyway"). He did not dispute the value of the reasoning engine; he relocated where it sits in the workflow.

Discussion

The evaluation supports our central claim: a description-logic ontology can represent conditional, graded, negatable EOL preferences and reason over them to produce a clinically honest output, including the honest non-answers (*no coverage*, *vague*) that a binary checkbox or a confident LLM cannot. The architecture we advocate is not ontology *versus* LLM but ontology *with* LLMs at the boundaries: the LLM extracts, the ontology is the auditable closed-world source of truth, and the reasoner produces the defensible four-category output. In high-stakes EOL care, a pure LLM is non-deterministic and prone to false confidence, whereas an auditable reasoner can honestly report when a directive is silent.

Expert review reframed, rather than refuted, the contribution. Three implications follow. First, **positioning**: ADO is a decision-support aid that helps pre-populate an ACP progress note and complements POLST and the code-status system; it is not a legal instrument, and presenting it as one would be both inaccurate and ethically inadvisable. The code-status mapping result (97% field agreement) is direct evidence that ADO can populate exactly those bedside instruments—and, via its conditionality flag, can carry information they cannot. Second, **dimensionality**: the current model represents only the *what-intervention* axis, but real preferences carry a second *who-decides* axis—patients frequently delegate or grant override authority to a surrogate. Modeling this axis (so that a clear "no intubation" paired with "do what my family says" does not resolve to a hard refusal) is our highest-priority extension. Third, **coverage**: artificial hydration/nutrition should be added as a sixth decision point, the POLST *time-limited trial* (a prospectively agreed, bounded trial to resolve prognostic uncertainty, distinct from a withdrawal threshold) should be modeled, and outputs should align with the real code-status taxonomy via an explicit *no-escalation* status.

Limitations. ADO output has no legal force, and the literature documents real limits to the value of advance directives in actual EOL decision making; the honest framing is that ADO surfaces one structured input into a relational, evidence-weighed process, not an oracle. OWL lacks native temporal reasoning, which we work around with numeric time

bounds rather than a general temporal logic. Negation under the open-world assumption has edge cases we continue to document. Encoding fidelity (how completely the pipeline captures intent, e.g., the LVAD/transplant qualifier in V9) is distinct from—and currently weaker than—reasoning fidelity (how correctly the reasoner evaluates what was encoded). Finally, the gold standards for the extraction and mapping evaluations were authored by the team—grounded in real template language and scored against independent POLST semantics, but team-authored nonetheless; two-annotator inter-rater agreement and a classmate fidelity study (documented vs. situational preferences) remain in progress, and validation against real-EHR code-status documentation (e.g., MIMIC-IV) is future work.

Conclusion

We built a computable ontology of EOL care preferences for advanced heart failure that reasons over conditional, graded, negatable preferences and returns a clinically honest four-category output, achieving 16/16 accuracy on a clinical vignette suite and demonstrating an end-to-end path from free-text directive to defensible decision via a closed-world LLM pipeline. Expert clinical-ethics review reframed ADO as a decision-support aid for advance-care-planning documentation—not a legal directive—and charted a concrete path forward centered on multidimensional (who-decides) preference modeling, additional decision points, and alignment with the bedside code-status system.

References

1. Silveira MJ, Kim SYH, Langa KM. Advance directives and outcomes of surrogate decision making before death. N Engl J Med. 2010;362(13):1211–8.
2. Yadav KN, Gabler NB, Cooney E, et al. Approximately one in three US adults completes any type of advance directive for end-of-life care. Health Aff. 2017;36(7):1244–51.
3. HL7 International. FHIR advance directive interoperability implementation guide. Release 1.1; 2024. Available from: https://hl7.org/fhir/us/pacio-adi/
4. Kremers FW, et al. Trends in the prevalence of implantable cardioverter-defibrillators in the United States. Heart Rhythm. 2023;20(5):698–706.
5. Allen LA, Stevenson LW, Grady KL, et al. Decision making in advanced heart failure: a scientific statement from the American Heart Association. Circulation. 2012;125(15):1928–52.
6. Periyakoil VS. Stanford letter project. Stanford School of Medicine; 2024. Available from: https://med.stanford.edu/letter.html
7. W3C OWL Working Group. OWL 2 web ontology language primer. 2nd ed. W3C Recommendation; 2012. Available from: https://www.w3.org/TR/owl2-primer/
8. Musen MA. The Protégé project: a look back and a look forward. AI Matters. 2015;1(4):4–12.
9. Lamy JB. Owlready: ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies. Artif Intell Med. 2017;80:11–28.
10. Criteria Committee of the New York Heart Association. Nomenclature and criteria for diagnosis of diseases of the heart and great vessels. 9th ed. Boston: Little, Brown; 1994.
11. SNOMED International. SNOMED CT; 2024. Available from: https://www.snomed.org/
12. National Hospice and Palliative Care Organization. CaringInfo: advance directives by state; 2024. Available from: https://www.caringinfo.org/
13. National POLST. POLST: portable medical orders; 2024. Available from: https://polst.org/

Appendix

A. Code, files, and how to run

All code is available at https://github.com/patrickwalsh26/bmds210-AD_Ontology. The repository contains:

- `advanced_directives.owl` — base OWL ontology (67 classes, 22 properties).
- `preference_input.py` — structured JSON → populated patient ontology pipeline.
- `query_evaluation.py` — scenario-based reasoner; 16 clinical vignettes.
- `code_status.py` — aggregates preferences into a hospital code status + POLST orders.
- `track3_evaluation.py` — code-status mapping evaluation over 12 directive profiles.
- `extraction_evaluation.py` — LLM extraction precision/recall over real directive text.
- `llm_extraction.py` — closed-world free-text directive → structured JSON (Claude).
- `demo.py` — six-scenario live demonstration.
- `populated_ontologies/` — example populated ontologies and input JSON.

Requirements: Python 3.8+, a Java runtime (for owlready2's HermiT reasoner), `pip install owlready2`; for the LLM pipeline, `pip install anthropic pydantic` and an `ANTHROPIC_API_KEY`.

Run the reasoner evaluation:

```
python preference_input.py --example      # generate example patient ontology
python query_evaluation.py                # run the 16-vignette suite
python track3_evaluation.py               # run the code-status mapping evaluation
python extraction_evaluation.py           # run LLM extraction precision/recall (needs API key)
python demo.py --pause                    # walk through the 6 demo scenarios
```

B. Screenshots of program execution

Figure/Listing 1 shows the reasoner's summary output; the per-vignette detail prints above it during execution.

Listing 1. `python query_evaluation.py` — summary output.

```
================================================================================
  SUMMARY
================================================================================
  Total vignettes      : 16
  Decision accuracy    : 16/16 (100%)
  Match type accuracy  : 16/16 (100%)

  By expected match type:
    clear          : 9/9 decisions correct, 9/9 match types correct
    no_coverage    : 2/2 decisions correct, 2/2 match types correct
    partial        : 4/4 decisions correct, 4/4 match types correct
    vague          : 1/1 decisions correct, 1/1 match types correct

  No failures.
```

Listing 2. `python track3_evaluation.py` — code-status mapping summary.

```
  SUMMARY
  Profiles               : 12
  Exact-profile agreement : 11/12 (92%)
  Code status            : 12/12 (100%)
  POLST A                : 12/12 (100%)
  POLST B                : 11/12 (92%)
  Overall field agreement : 35/36 (97%)
  (code-status confusion matrix fully diagonal; sole divergence P5 discussed in text)
```

Listing 3. `python extraction_evaluation.py` — LLM extraction summary.

```
  Precision : 0.94    Recall : 1.00    F1 : 0.97
  Per-field on 15 matched preferences: negation 15/15, type 15/15, conditionality 15/15
  Over-extracted/hallucinated (out-of-scope FP) : 0   <- closed-world discipline holds
```

C. Team member contributions

D. Statement on sharing

We consent to the teaching team sharing this project as an example in future offerings of the course, with identifying information removed.

Member	Project contributions	Report contributions
Patrick Walsh	Ontology design and implementation in Protégé; concept inventory; structured preference-input pipeline; SNOMED CT grounding; LLM extraction pipeline; expert-review coordination	Primary author; wrote all sections; compiled references
Darren Chan	Repository setup and maintenance; scenario-based query engine; clinical vignette design; testing and gap analysis	Reviewed and edited; contributed evaluation results and analysis